Contain, Conform, Compare: A Red-Teamed Kit for Evaluating Small Coding Agents on One GPU

Attacked per-job containers, token-level prompt checks, and a pre-registered test of four prompt divergences

Nguyen Minh Thang
VNU University of Science, 334 Nguyen Trai, Thanh Xuan, Hanoi, Vietnam

November 2026

Abstract

A hosted score in the Gemma 4 Developer Agent competition [1] is one noisy number, so development needs a local evaluator that is fast, safe for agent-written code, and matching the hosted scorer's prompts. `drydock-eval` evaluates a small coding agent (Gemma 4 E4B [2]) at about 250 runs per hour on one RTX 5090. *Contain* runs each job in a fresh container without network and passed 200 scripted escape tests; *conform* provides eight checks that compare a local stack's requests with the scorer's configuration, down to rendered tokens; *compare* provides paired statistics that respect repository clusters. AI agents built and attacked the kit; a human owner approved machine use and release. Across seven runner versions no attack escaped the container's namespaces or privileges, yet nine demonstrated high-severity vectors came through what the host shared with it, in designs passing their own escape tests; the last independent verifier found no high or medium issue. Rendered prompts matched the Kaggle template on 396 of 396 requests by token count and 50 of 50 by token id. A pre-registered five-arm experiment (231 tasks, 4,620 jobs) measured four prompt divergences: no E4B solve-rate change was detected (largest estimate -2.1 points, 95% interval -4.4 to $+0.4$).

1 Introduction

The competition scores an agent bundle on hidden GitHub-issue tasks, with Gemma 4 served by vLLM [11]. Third-party analyses put the noise of one hosted score near 1.5 of 58 tasks [3, 4]. Local evaluation must therefore carry development; on a Kaggle kernel with four L4 GPUs our stack finished 35.2 agent runs per hour, serially.

Safety. An agent run executes model-written shell commands and installs model-edited packages; our first local setup ran them with no sandbox.

Fidelity. We call a *rendering trap* a configuration detail that silently changes the token sequence the model receives while runs still finish and score. The *scorer path* is the scorer's configuration reproduced locally. In our first setup the served model name contained "gemma4", so the agent framework (google-adk 1.36.1) sent tool results with role `tool_responses` instead of `tool` [5]. Prompts rebuilt from its traces matched the server's token counts on 106 of 114 calls only when rendered with `tool_responses` (0 of 114 with `tool`). Solve counts hid it: that setup solved 3 and 5 of 24 tasks against 4 and 4 on Kaggle (Fisher p 0.74 and 0.75).

These traps are public: the role rule [5], thought handling in history [6, 7, 8], a newline after thinking text [9], and the scorer's compaction settings [10]. We discovered none of them; we

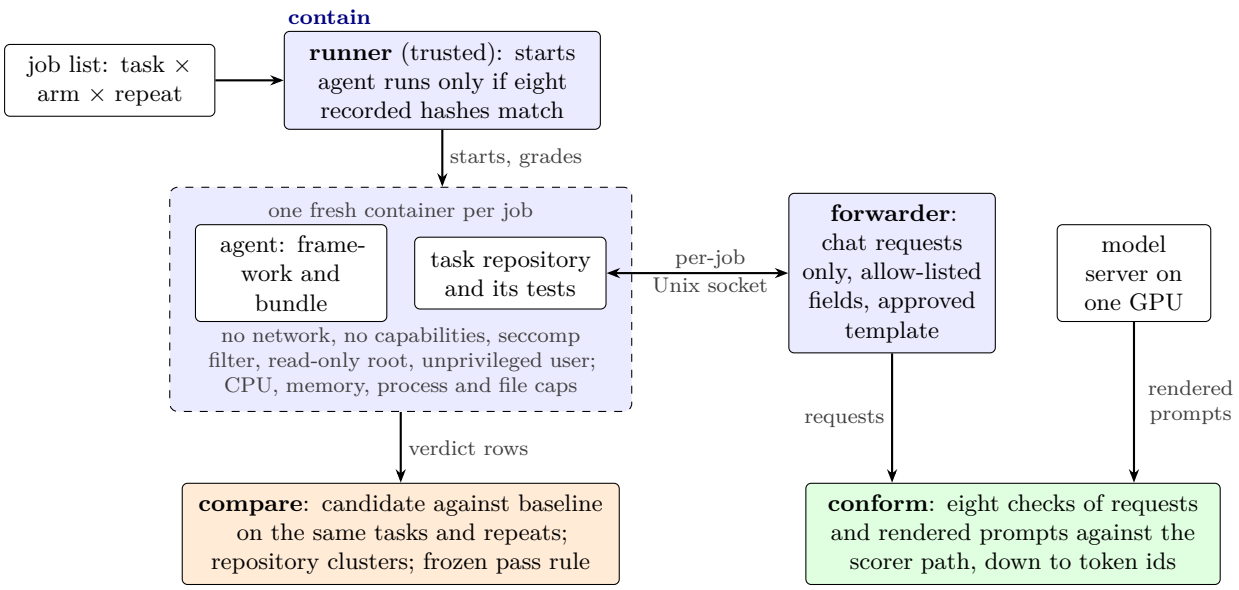


Figure 1: The kit: where each module sits.

contribute checks that detect them (§4) and a measurement of three of them plus a template swap (§6).

2 Scope, threat model and rules

Stack. Gemma 4 E4B, 4-bit quantization-aware checkpoint (Kaggle model version 2); vLLM 0.19.1; google-adk 1.36.1; the competition's harness packages. Tasks come from Berth [12]: 579 of its 653 tasks load in the harness and pass our validation (the reference patch passes, an empty patch fails). Experiments ran on a shared single-GPU workstation (RTX 5090, 32 GB).

Threat model. The adversary is the code the agent writes. It may try to read host files or other jobs, reach the network, exhaust host resources, forge its verdict, or outlive its job. Kernel exploits are out of scope.

Who did the work. AI agents wrote the kit's code, attacks and analyses (builders). Separate AI agents (verifiers) re-implemented or re-ran the work adversarially. A lead AI session froze the protocols; a human owner approved isolation, machine time and release. "Verified" below means this process, not human review.

3 Contain: a runner and the attacks it failed

In the final design each job gets one fresh container: no network, no capabilities, a seccomp filter, a read-only root, an unprivileged user, 2 CPUs, 10 GiB of memory, 4,096 processes, and temporary filesystems capped in bytes and inodes. The only request channel is a per-job Unix socket to a forwarder, which passes only chat-completion requests with allow-listed fields, text-only content and an approved template hash, under rate and size caps. Agent runs start only if eight recorded hashes (image, code, flags, model files) still match.

A *canary* is a scripted attack with a machine-checked outcome.

Passing canaries proved little. No attack escaped the container's namespaces or privileges in any round. Nine demonstrated high-severity vectors came through what the host shared with it:

Table 1: Runner versions.

Runner version	Canaries passed	Security findings of independent verifiers
1. Container per command	52/52	4 high: host-side install ran the agent's `setup.py` (builder-found); symlink leaked a host file; 10 GiB of host disk reserved in 0.005 s; host copied a sparse file in full
2. Hardened, per command	60/60 (57/60 before canary fixes)	4 high, 1 untested: host overwrote a file through a planted link; cleanup followed links; 6 GiB reserved in 0.012 s; 200,000 files in 2.4 s
3. Container per job	98/98	1 high: the model server could fetch URLs named in requests; 2 medium
4–6. Request filter added, tightened	134/134, 159/159, 191/191	medium at most
7. Final	200/200	no high or medium

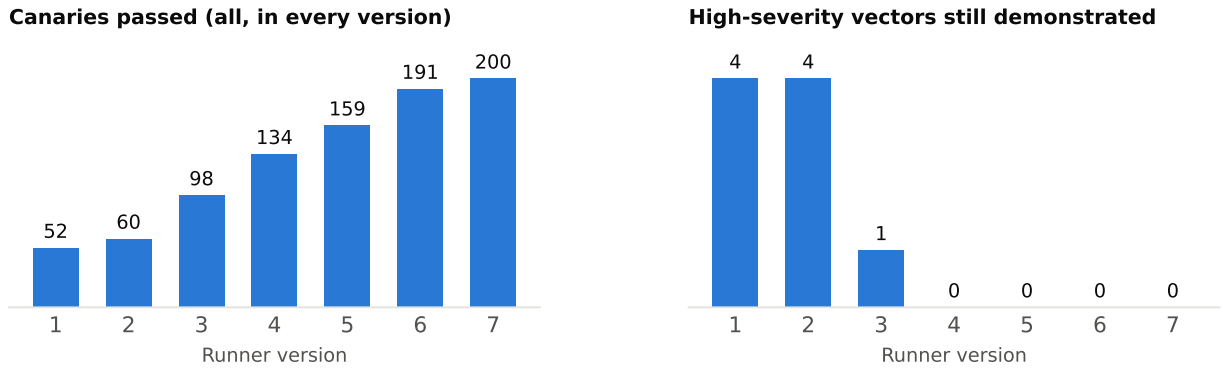


Figure 2: Canaries passed and high-severity vectors found, per runner version.

host-side steps that touched agent-written files, resource limits (a known class [13]) and the model server.

Final version. Controls on 481 tasks (reference patch 481/481, empty patch 0/481) showed no verdict, exit-code or test-summary difference from the previous version. After the runner was killed, the container was gone within 149 s with no further model request. The final runner completed a 24-job list at 251–254 runs per hour with 8 workers.

What remains. Containers share the host kernel; user namespaces are not remapped; the request parser runs in the trusted runner; a job can slow others through the shared model server within its caps. Our claim is conditional: against the stated adversary and with the eight hashes matching, no tested attack succeeded.

4 Conform: eight checks against the scorer path

`conform` reads the requests a local stack sent and, where logged, the prompts the server rendered. Reference values come from the public starter notebook [10], our Kaggle-kernel traces, and the Kaggle model's chat template.

Role. From each google-adk release's source (only 1.36.1 was executed): 1.34.0–1.39.1 and 2.0–2.4 send `tool_responses` when the model string contains "gemma4"; 2.5–2.11 when it matches `gemma-?4`; 2.12–2.14 exempt three OpenAI-compatible providers [5]. The harness packages require `>=1.34.0,<2.0`.

Table 2: The eight checks.

Check	Compares	Mutation runs caught
role	role of tool results	53/53
sampling	temperature, top-p, token and thinking budgets	52/52
tools	declared tool schemas	20/20
shape	message skeleton, system text, field encoding	72/72
template	rendered prompt against the Kaggle template	13/21 by token count, 24/24 by token id
thoughts	history thoughts, under the key the server reads	52/52
newline	newline before the token closing a thought	12/12
compaction	when history is summarized (14,336 tokens [10])	19/19

Thoughts. vLLM 0.19.1 reads an earlier assistant thought only from the field `reasoning`. google-adk 1.36.1 writes `reasoning_content`, which that server drops silently [8]; the competition harness writes both. Removing `reasoning` in a probe lost 55 of 55 thoughts from the rendered prompt.

Template. The check re-renders each request outside vLLM with the Kaggle template and compares token counts with the server's, or token ids when the server logs them, because identical strings can tokenize differently [16].

Evidence. In version 2's 24-task parity run, all 2,566 tool results had role `tool`, and re-rendering reproduced the server's token count on 396 of 396 requests (with `tool_responses`: 0 of 358 multi-turn requests). With the final runner, the checker reproduced the token ids of 243 of 243 returned calls, and a renderer written separately by the verifier matched 50 of 50 scorer-path requests. Prompts rebuilt from early calls of our Kaggle-kernel traces match the recorded counts on 181 of 189 calls with role `tool` and thoughts kept, 1 of 189 with `tool_responses` and 0 of 189 with thoughts dropped: our only Kaggle-side evidence, and count-level.

Mutation suite. 103 mutations (85 drifts, 18 benign controls) in four scripted scenarios against a mock server give 412 runs. The intended check caught 317 of 325 applicable drift runs; the 8 misses are same-length template changes seen through counts only. No check failed in 72 control runs (24 warned).

5 Compare: a decision rule that respects repositories

Tasks from one repository succeed and fail together. Clustered inference for model evaluations is prior art [19]; `compare` packages a small-cluster version with a frozen pass rule and its simulated behaviour.

Candidate and baseline run on the same tasks and repeats. A candidate passes only if (1) the mean gain is at least 2/58 (3.45 points, two leaderboard tasks); (2) one-sided $p \leq 0.025$ under a null-imposed wild cluster bootstrap-t over repositories, with Rademacher weights and 9,999 draws [20, 21]; (3) at least three repositories gain and every leave-one-repository-out estimate is positive; (4) agent time stays within 1.2 times baseline and infrastructure errors do not rise; and (5) no repository loses two task-equivalents or more. Two panels were frozen before any outcome: a small one (72 tasks, 18 repositories) and a large one (200 tasks, 20 repositories).

The 2.5% level holds for exchangeable effects, not for skewed ones, and not if tasks are dropped after unblinding. Independent re-implementations matched the rule code on 280 of 280 clean datasets (first version) and, with rule readings aligned, 193 of 194 (amended). Applied three times, to screen leaders from one separate 48-task set (25, 22, 25 of 96 runs solved; baselines 19, 21, 20);

Table 3: Simulated pass rates (3,000 simulations per cell; 400 for the dropping row).

Simulated scenario	Small panel	Large panel
No effect, exchangeable tasks	0.23–1.90%	0.00–0.50%
No mean effect, candidate breaks 20% of repositories	7.50%	8.43%
No effect, worst tasks dropped after seeing outcomes	31.75%	40.75%
Even gain of 3/58 (power)	≈60%	≈90%
Gain confined to four repositories (power)	15.9%	24.9%

Table 4: Arms; changes in percentage points.

Arm	Difference from the scorer path	Requests changed by the divergence	Change against F [95% interval]; Holm p
F	none: role `tool`, Kaggle template, compaction at 14,336 tokens, thoughts under both keys	—	solve rate 23.8%
R	model name contains "gemma4": tool results get role `tool_responses`	89%	−0.3 [−2.8, +2.2]; p = 1
C	compaction unset: prompts grow until rejected above 24,576 tokens (the run ends; edits are still graded)	21%	+0.2 [−2.0, +2.5]; p = 1
H	history thoughts removed from every request, as when a client omits the key the server reads	90%	−2.1 [−4.4, +0.4]; p = 0.44
T	the Hugging Face E4B template: a thought opener is forced after each tool result	85%	+0.8 [−1.4, +2.9]; p = 1

none passed. Gains: −0.35 points on the small panel (p = 0.60); on the 128 large-panel tasks outside it, −0.39 (p = 0.60), then +4.10 (p = 0.04 > 0.025, uncorrected). Condition (5): first application only.

6 Case study: four prompt divergences on E4B

Five arms share one model server, one withheld agent bundle, google-adk 1.36.1 and all sampling settings.

No newline arm was run: both templates render it identically on this server.

Panel. Removing tasks used by other panels, reserved repositories, slow tasks and duplicates from the 579 leaves 231 tasks in 15 repositories. They run 4 repeats in 5 arms (4,620 jobs); for each task and repeat the arms run back to back in counterbalanced order.

Analysis, frozen before any outcome (protocol sha256 `070c9655d96372d054f430727259de7b25af2ca93c3ec4c9295dfdc37aeb8cf4`). The estimand d_X is the mean over tasks of arm X's solve rate minus arm F's. The primary test is an exact two-sided task-level sign-flip test, Holm-corrected over the four arms at 0.05; only then do we say that X raises or lowers E4B's solve rate *on these tasks*. Intervals (unadjusted) invert that test. A repository-cluster bootstrap interval is secondary (lower simulated power). A frozen checker re-renders every returned request from the job's dump, matches the server's token count and confirms the job's arm; a failing job is re-run. Outcomes stayed blind until a pre-registered cut-off (2026-11-11 06:00 UTC) and were unblinded once.

Power, simulated before data. The family-wise false-claim rate is 1.2–4.2%. At 231 tasks

an even 5-point drop in one arm is detected with probability 82–100% across the simulated settings; the drop or gain detected with 80% power ranges from below 3 to about 5.2 points.

Deviations recorded blind. Independent validation of the final runner finished only after launch. From about an hour in, a second workload held about 9.6 GiB of GPU memory, so timings are under mixed load. The block stopped three times: after 967 jobs at the runner's open-file limit (cause unknown), after 378 more by an external termination signal on the shared machine, and after 3,074 more when the kernel's out-of-memory killer ended the model server (cause undiagnosed). Each resume ran only never-started jobs; the 24 interrupted jobs were re-run once.

Results. All jobs finished before the cut-off. Every arm has a valid row in 924 of 924 slots, and the checker passed all 4,620 analysed jobs. No pre-emption was recorded; 6.0% of runs took over 240 s, above the pre-registered 5% threshold. No difference from F was detected for any arm (Table 4, Figure 3B; smallest Holm-adjusted p 0.44); the secondary repository-cluster intervals all include zero. The largest estimate is H's −2.1 points (unadjusted p 0.11; −1.3 with time-limit rows as missing). In exploratory measures, H runs averaged 137 s against 78 s and ended unsolved at the bundle's five-minute limit in 11.5% of runs against 1.8%; in C, 12.3% of runs ended unsolved on a rejected over-long prompt. These are null results, not evidence of no effect: the simultaneous 98.75% intervals allow a drop of 5.1 points (H) and a gain of 3.5 (T).

7 Related work

SandboxEval [14] and SandboxEscapeBench [15] test sandboxes and models' ability to escape them.

Several paper-track resources overlap this one (described from their public text only): LAB [22], a Docker sandbox with 11 tests and a bundle gate with 16 detectors validated by fault injection; Szeto et al.'s cloud rig with 141 run folders [23]; gemma4-swe-kit [17], a proxy reproducing the scorer's template, call parsing and compaction on other servers; a dataset card reporting count parity on 107 captured vLLM requests [18]; Silent Zeros [24], a bundle checker tested on 22 broken bundles; and ThoughtKeep [25], 1,470 Gemma 4 31B attempts on thought retention. We did not find in those texts, and offer here: a runner attacked by independent verifiers with its canary misses published, token-id comparison backed by a mutation suite, and a trap measurement on E4B with a per-job prompt check.

8 Limitations

- **Solve-level parity is not refuted, not established.** The evidence (Figure 3A) is one 24-task list, one repeat per run; ten runs solved only six distinct tasks.
- **Kaggle-side facts are partly inferred**: the hosted google-adk release, tool-schema equality, and the identity of the hosted template and weights with the public Kaggle model.
- **Environment differences remain.** One RTX 5090 instead of four L4s; 2 CPUs and 10 GiB per job; a bare `pytest` command, used in 30 of 216 commands in our Kaggle traces, is absent locally.
- **The case study** covers one model, one bundle, one google-adk release and one configuration per divergence. Arm T swaps a whole template; in arm H thoughts still reach compaction summaries.

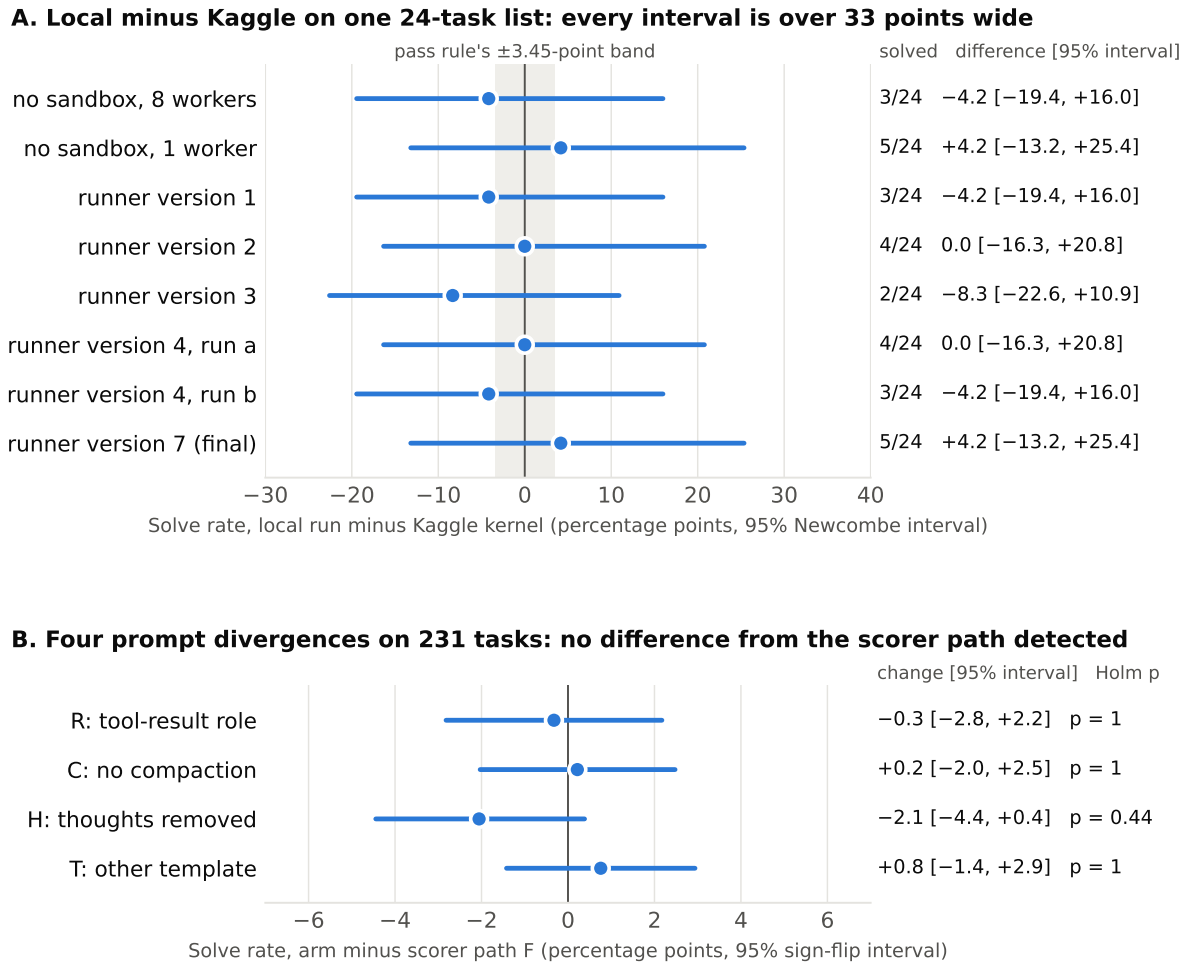


Figure 3: (A) Solve-rate difference, local minus Kaggle (pooled 8/48), for eight local runs of one 24-task list (95% Newcombe intervals, runs treated as independent); shaded: the pass rule's ±3.45 points. (B) Change in solve rate against F on 231 tasks: estimates, 95% intervals and Holm-adjusted p.

9 Availability

`drydock-eval` 0.4.0 (Apache-2.0) has three modules: `conform` (checks, mock server, mutation suite), `contain` (a reference runner tree, its escape tests, a gate helper) and `compare` (pass rule, sign-flip tests, simulations). Its 435 CPU tests pass. The released runner has not been run in a container as released: the 200 of 200 result is for the deployed tree, of which the release is an identifier-substituted copy with two canaries rewritten. Code: https://github.com/MinhThanghus/drydock-eval (tag `v0.4.0`; branch `paper`: supplement, §6 outcome rows). The package also carries copies of the §6 protocol, analysis code and job-list generator, redacted where needed, each with its original's sha256; the agent bundle and request dumps are planned for 2026-12-03.

References

[1] Google DeepMind. Gemma 4 Developer Agent Competition. Kaggle, 2026.

[2] Gemma Team. Gemma 4 Technical Report. arXiv:2607.02770, 2026.

[3] dariushafshar. Gemma 4 Agent: Public Config Draws. Kaggle, 2026.

[4] Mobassir. Too Small to See. Kaggle, 2026.

[5] rladeira. google/adk-python issue #7977. 2026.

[6] Google. Gemma 4 Prompt Formatting. 2026.

[7] dimondev. Hugging Face gemma-4-E2B-it pull request #44. 2026.

[8] delta9000. vLLM issue #38488. 2026.

[9] G. Gerganov. Hugging Face gemma-4-31B-it discussion #135. 2026.

[10] ryanholbrook. Getting Started - Gemma 4 Developer Agent. Kaggle, 2026.

[11] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.

[12] J. Patel. Berth. Kaggle, 2026.

[13] X. Gao et al. Houdini's Escape. CCS 2019.

[14] R. Rabin et al. SandboxEval. arXiv:2504.00018, 2025.

[15] R. Marchand et al. Quantifying Frontier LLM Capabilities for Container Sandbox Escape. ICML 2026.

[16] Agent Lightning Team. No More Retokenization Drift. 2025.

[17] damsolanke. gemma4-swe-kit. GitHub, 2026.

[18] tyfeng1997. gemma4-e4b-swe-sft-32k-competition-template. Hugging Face, 2026.

[19] E. Miller. Adding Error Bars to Evals. arXiv:2411.00640, 2024.

[20] A. Cameron, J. Gelbach, D. Miller. Bootstrap-Based Improvements for Inference with Clustered Errors. REStat, 2008.

[21] J. MacKinnon, M. Webb. Wild Bootstrap Inference for Wildly Different Cluster Sizes. J. Applied Econometrics, 2017.

[22] marcofpereira. Gemma 4 Developer Agent LAB. Kaggle, 2026.

[23] J. Szeto, D. Guo, H. Szeto. Scorer-Faithful Local Evaluation for Small Coding Agents. Kaggle, 2026.

[24] houchiunyo. Gemma 4 agent resource: runs & checker. Kaggle, 2026.

[25] hemantkumarbk. Coding-Agent Thought Retention: Reproduction Data. Kaggle, 2026.